STUDY · NETWORK

# HTTP는 왜 1.1에서 2, 3까지 진화했을까?

문제 → 해결 → 새로운 문제의 반복

HTTP/1.1 → HTTP/2 → HTTP/3

8기 BE 스타크

AGENDA

# 오늘 볼 흐름

01 웹은 왜 HTTP 성능 개선을 필요로 했나

02 HTTP/1.1의 특징과 한계

03 HTTP/2가 해결한 문제

04 HTTP/2에도 남은 문제

05 HTTP/3와 QUIC의 등장

06 HTTP/1.1 · 2 · 3 비교 정리

핵심: **이전 버전의 한계가 다음 버전의 등장 이유가 되었다.**

왜 성능이 문제인가

# 웹페이지 하나는 요청 하나가 아니다

**Browser**

한 번의 페이지 로드



- index.html
- style.css
- app.js
- image.png
- font.woff
- api/users

**한 페이지 로딩 = 수십 ~ 수백 개의 HTTP 요청**

PART 01

# HTTP/1.1

## 연결 재사용으로 비용을 줄이다

...하지만 한 연결 안의 기다림이 남는다

HTTP/1.1 · 핵심

# 매번 연결하지 말자: Persistent Connection

### Before · HTTP/1.0 — 요청마다 새 연결

TCP 연결 → Request 1 → 응답 → 종료

TCP 연결 → Request 2 → 응답 → 종료

TCP 연결 → Request 3 → 응답 → 종료

### After · HTTP/1.1 — 연결 하나를 재사용

TCP 연결 1개

→ Request 1

→ Request 2

→ Request 3

매 요청마다 3-way handshake 비용을 치르지 않는다.

HTTP/1.1 · 한계

# 한 연결 안에서는 기다림이 생긴다

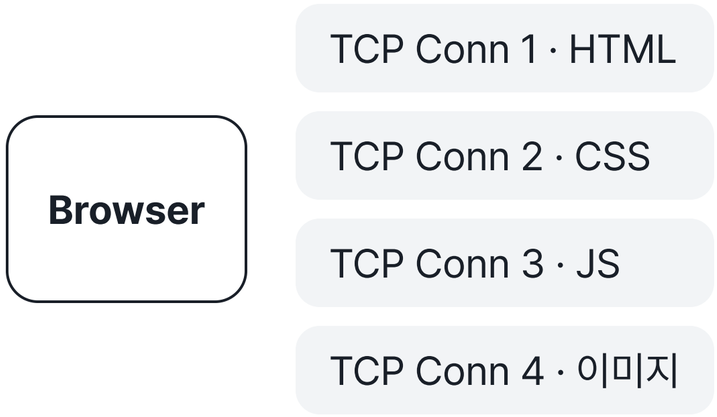
하나의 연결 · 처리 순서 →

| 큰 이미지 응답 (처리 중...) | CSS · 대기 | JS · 대기 | Font · 대기 |
| --- | --- | --- | --- |

**Head-of-Line (HOL) Blocking** 앞이 늦어지면 뒤도 막힌다.

HTTP/1.1 · 우회

# 그래서 연결을 여러 개 열었다




**장점**

한 연결이 막혀도 다른 연결은 진행 가능

**단점**

연결마다 TCP 비용 · 서버 리소스 증가 · 네트워크 혼잡

완전한 해결이 아니라 **우회**에 가깝다.

HTTP/1.1 · 정리

# 성과와 남은 한계

## 성과

· TCP 연결 재사용
· 매 요청 연결 생성 비용 감소
· 웹 성능 개선

## 한계

· 한 연결 안 순서 대기 (HOL Blocking)
· 우회로 여러 TCP 연결 사용
· 연결 수 증가 → 서버·네트워크 부담

다음 질문

**하나의 연결만 쓰면서 여러 요청을 동시에 처리할 수는 없을까?**

PART 02

# HTTP/2

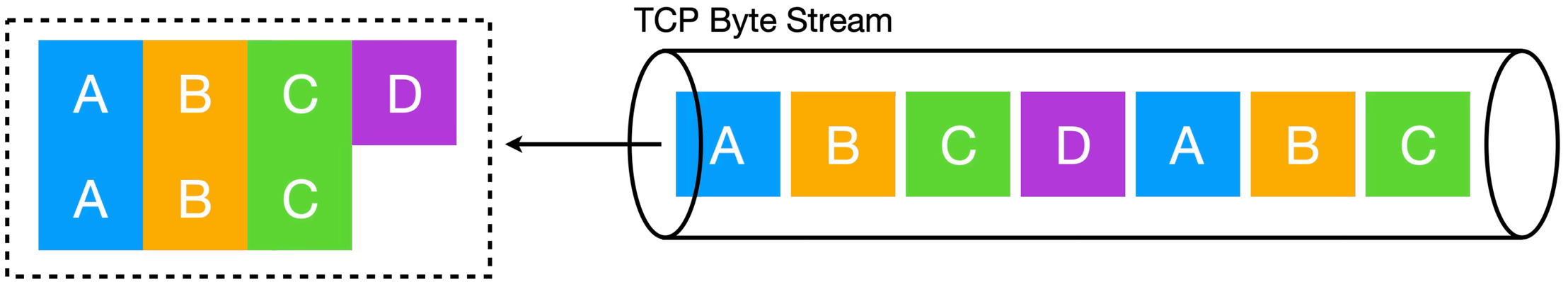
## 하나의 연결에서 동시에 처리하다

...하지만 TCP의 한계는 그대로 남는다

HTTP/2 · 핵심

# 하나의 연결에서 동시에: Multiplexing

Stream Frame Multiplexing

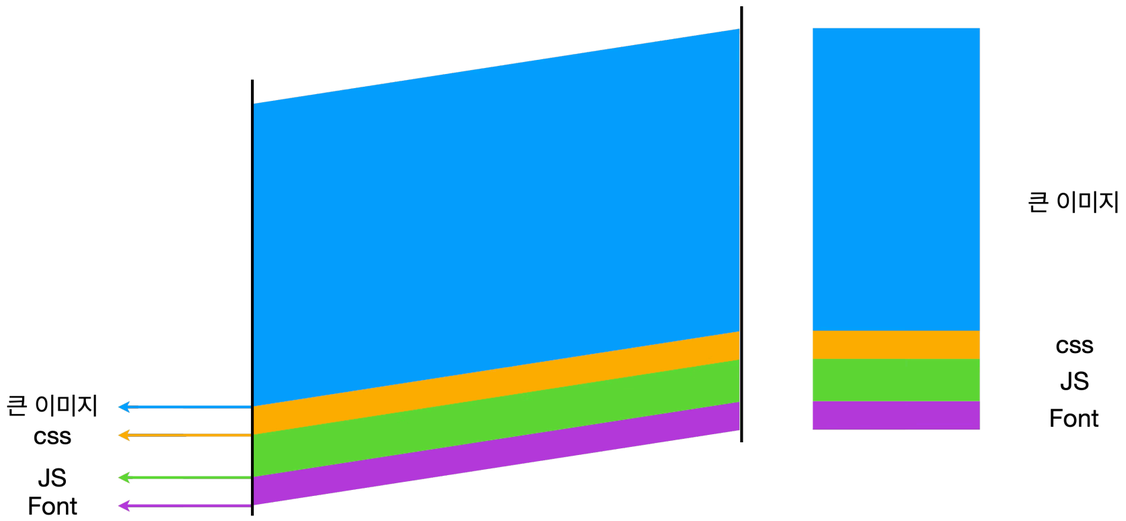



여러 Stream의 Frame을 하나의 연결에 섞어 전송 → 받는 쪽에서 Stream별로 재조립

HTTP/2 · 효과

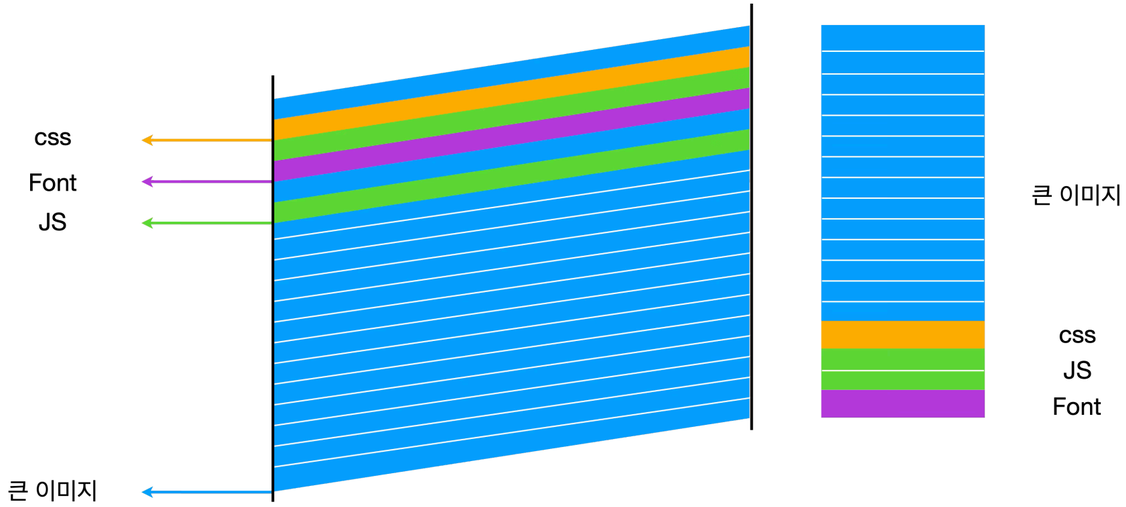
# HTTP 계층의 HOL Blocking을 줄였다

### HTTP/1.1 — 큰 이미지가 파이프를 통째로 막는다




css · JS · Font가 큰 이미지 뒤에서 순서대로 대기

### HTTP/2 — Frame으로 쪼개 사이사이 끼워 보낸다




큰 이미지가 Frame으로 나뉘어 작은 응답이 먼저 도착

큰 응답이 진행 중이어도 작은 응답의 Frame을 중간에 끼워 보낸다.

HTTP/2 · 추가 개선

# Header도 압축했다: HPACK

### HTTP/1.1 — 매번 전체 반복

Cookie · User-Agent · Accept · Authorization

Cookie · User-Agent · Accept · Authorization

Cookie · User-Agent · Accept · Authorization

### HTTP/2 — 공통 Header는 압축·참조

Cookie · User-Agent · Accept · Authorization

↑ 참조 (ref)

↑ 참조 (ref)

반복 Header 전송 비용 감소 → 요청이 많을수록 이점 증가.

HTTP/2 · 정리

# 성과와 남은 의문

### 개선

· 하나의 TCP 연결 사용
· 여러 Stream 동시 처리 (Frame 기반)
· Multiplexing · HPACK 압축

### 해결한 문제

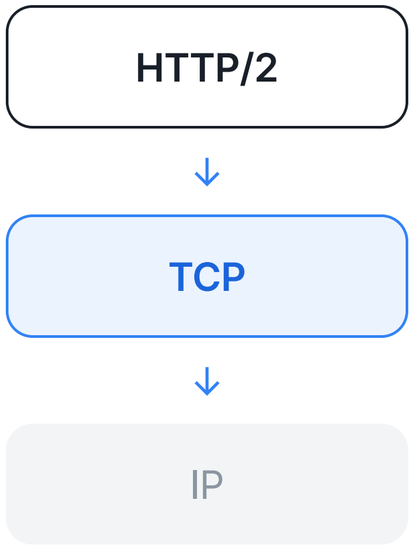
· 순서 대기(HOL) 완화
· 여러 TCP 연결 부담 감소

다음 질문

**HTTP/2는 정말 HOL Blocking을 완전히 해결했을까?**

HTTP/2 · 남은 문제

# HTTP/2는 여전히 TCP 위에서 동작한다




### TCP의 특징

· 순서 보장 · 신뢰성 보장

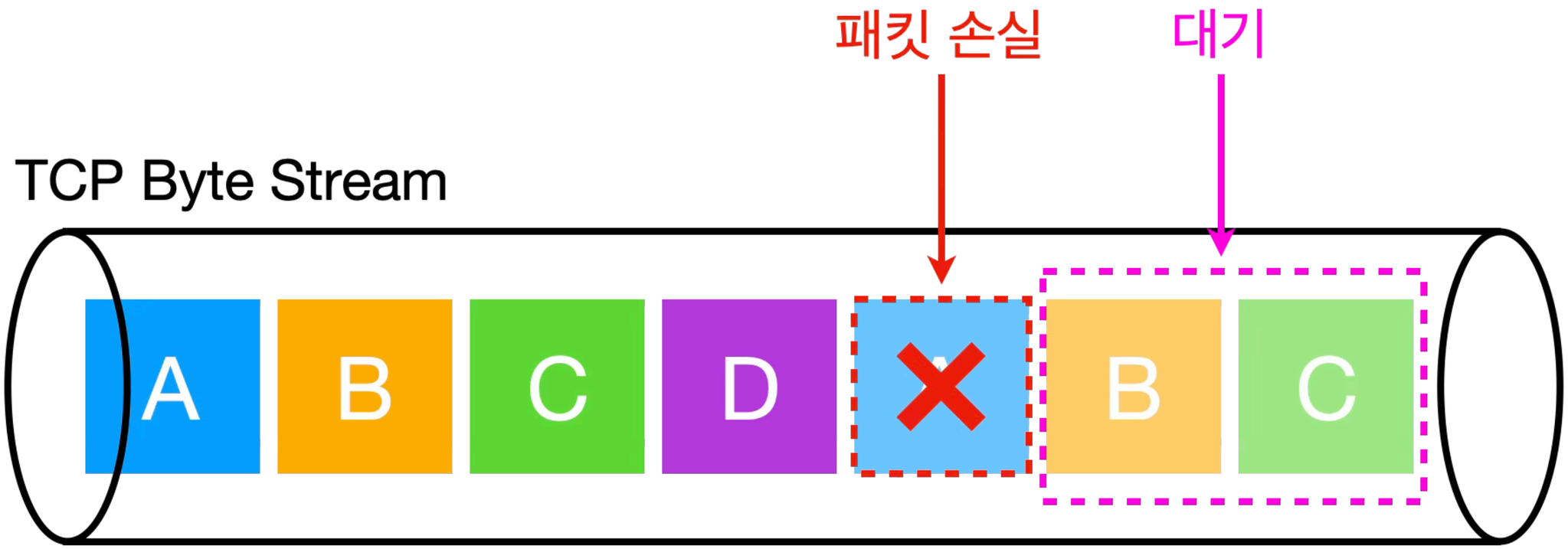
· 패킷 손실 시 재전송

· **앞 데이터가 손실되면 뒤 데이터도 애플리케이션에 전달되지 못함**

HTTP/2엔 Stream이 여럿이지만, TCP엔 **하나의 긴 바이트 스트림**일 뿐.

TCP 계층 HOL Blocking

# 패킷 하나가 막히면 모든 Stream이 멈춘다




손실된 패킷이 재전송될 때까지 — 뒤의 B·C가 멀쩡히 도착해도 전달되지 못한다

HTTP/2의 한계는 HTTP/2 자체가 아니라, 그 아래 **TCP의 특성**에서 온다.

HTTP/2 · 한계 정리

# 해결했지만, 남긴 것도 있다

### 해결한 것

· HTTP 계층 HOL Blocking
· 여러 연결 사용 부담
· Header 반복 전송 비용

### 남은 것

· **TCP 계층 HOL Blocking**
· 패킷 손실 시 전체 Stream 영향
· TCP + TLS 연결 수립 비용

다음 질문

**TCP가 문제라면, HTTP가 TCP를 벗어날 수는 없을까?**

PART 03

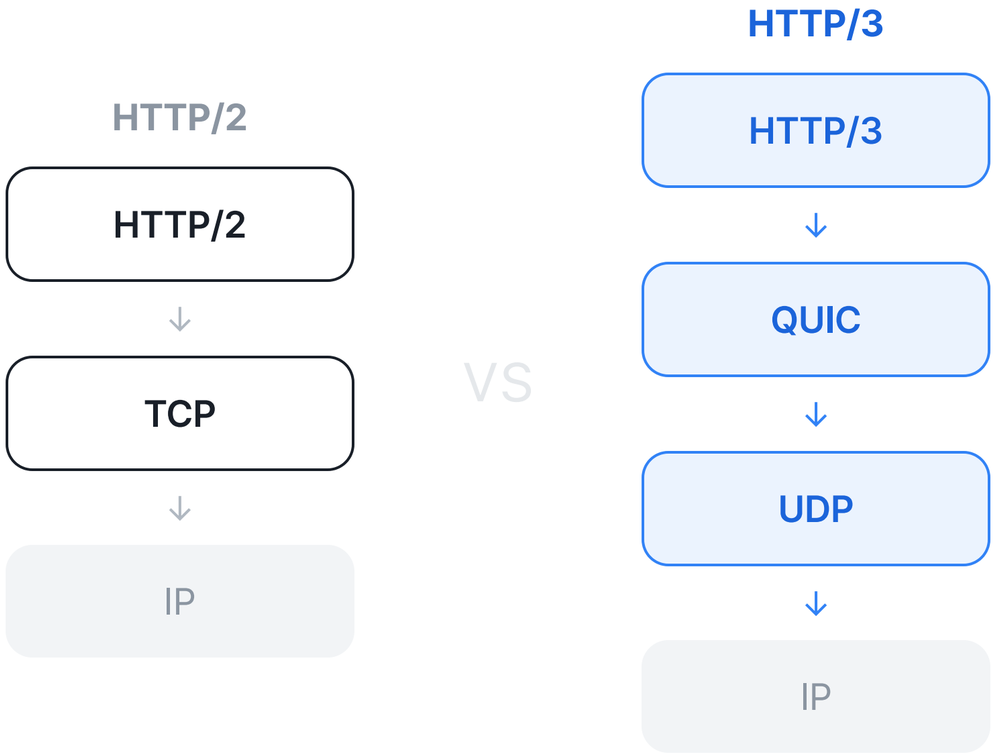
# HTTP/3

## 전송 계층을 바꾸다: QUIC

TCP를 벗어나 Stream 독립성을 얻다

HTTP/3 · 핵심

# TCP 대신 QUIC over UDP



HTTP/3 · QUIC

# UDP인데 신뢰성은 어떻게?

**UDP가 안 주는 것**

연결 관리 · 재전송

순서 보장 · 흐름 제어 · 혼잡 제어

→

**QUIC이 직접 제공**

Stream · 재전송 · 흐름 제어 · 혼잡 제어

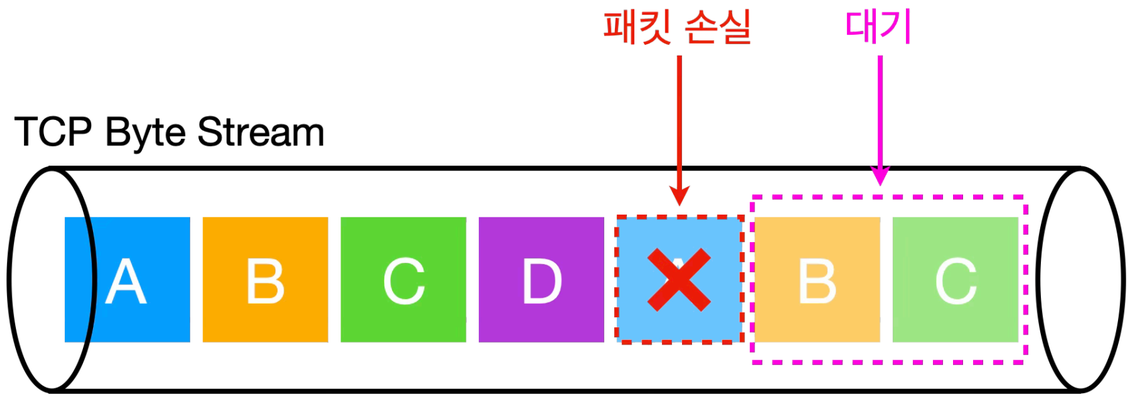
**TLS 1.3 암호화 · 빠른 연결 수립**

UDP를 쓴다고 신뢰성을 포기한 게 아니다 — QUIC이 필요한 기능을 직접 구현한다.

HTTP/3 · 효과

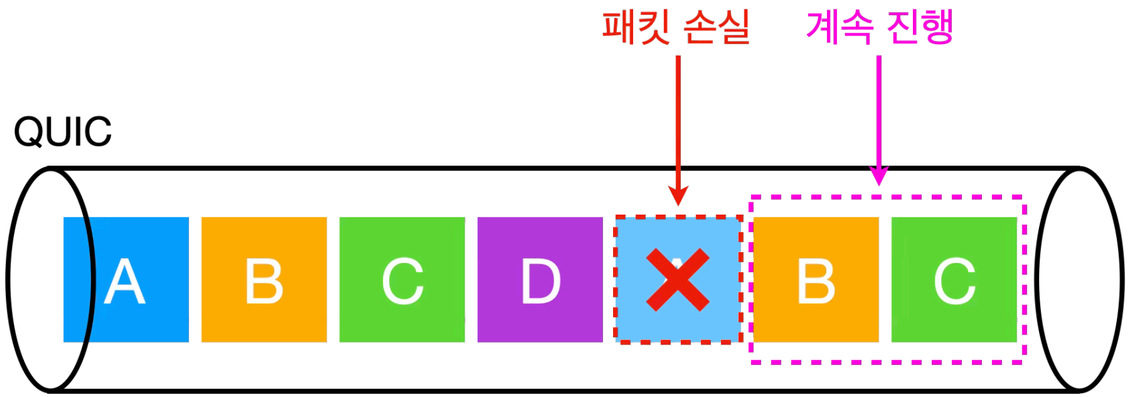
# Stream 단위로 독립적이다

### HTTP/2 over TCP — 한 줄에 갇힘




A 손실 → B·C도 멈춤

### HTTP/3 over QUIC — Stream 독립




A만 대기 → B·C는 계속 진행

HTTP/3 · 연결 수립

# 연결 시작도 빠르게: TLS 1.3 통합

**TCP + TLS**

SYN → SYN/ACK → ACK → TLS Handshake → HTTP

**QUIC**

QUIC + TLS 1.3 통합 Handshake → HTTP

**재방문 시 0-RTT**

전송 중 병목뿐 아니라 **연결 시작 단계의 지연**도 줄인다.

HTTP/3 · 정리

# HTTP/3가 해결한 것

## 핵심

· QUIC · UDP 기반
· Stream 독립성
· TLS 1.3 통합

## 해결한 문제

· TCP 계층 HOL Blocking 완화
· 연결 수립 비용 감소
· 네트워크가 자주 바뀌는 환경에 강함

COMPARISON

# HTTP/1.1 vs HTTP/2 vs HTTP/3

| 구분 | HTTP/1.1 | HTTP/2 | HTTP/3 |
|---|---|---|---|
| 기반 전송 | TCP | TCP | **QUIC over UDP** |
| 연결 사용 | 여러 TCP 연결 | 1 TCP · Multiplexing | 1 QUIC · Multiplexing |
| 데이터 단위 | 메시지 중심 | Frame 중심 | Frame + QUIC Stream |
| Header 압축 | 없음 | HPACK | QPACK |
| HTTP 계층 HOL | 있음 | 크게 완화 | 완화 |
| TCP 계층 HOL | 있음 | 있음 | TCP 미사용 → 완화 |
| 연결 수립 | TCP + TLS | TCP + TLS | **QUIC + TLS 1.3 통합** |

오해 정리

# HTTP/2가 해결했는데, 왜 HTTP/3가 필요했을까?

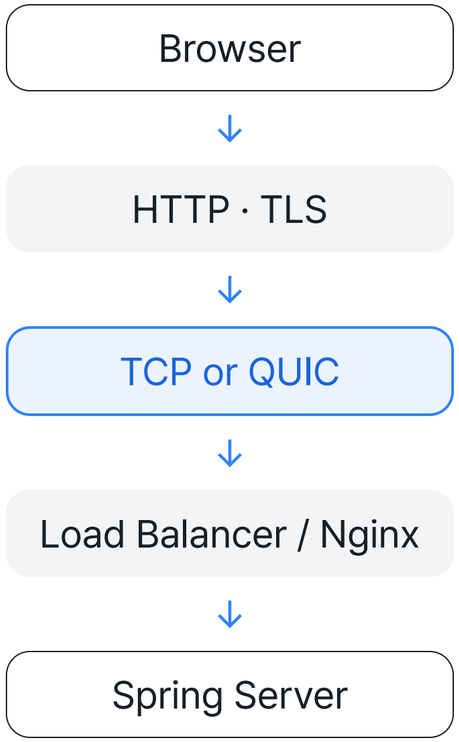
✓ **HTTP 계층 HOL Blocking**
HTTP/2의 Multiplexing이 해결

✕ **TCP 계층 HOL Blocking**
HTTP/2도 TCP 위에서 동작 → 패킷 손실 시 그대로 남음

→ **HTTP/3 · QUIC**
바로 이 TCP 계층 문제를 풀기 위해 등장

WHY IT MATTERS

# 백엔드 개발자가 왜 알아야 할까




· API 응답 지연 원인을 더 넓게 볼 수 있다

· HTTP/2 · 3 설정의 의미를 이해할 수 있다

· Nginx · ALB · CDN 설정 이해에 도움

· "빠르다"가 아니라 **"언제 유리한지"** 판단할 수 있다

SUMMARY

# 정리

### HTTP/1.1

연결 재사용으로 비용 절감.
한 연결 안 HOL Blocking 남음.

### HTTP/2

Frame · Multiplexing.
HTTP 계층 HOL 완화, TCP 계층은 남음.

### HTTP/3

QUIC over UDP.
Stream 독립성 · 연결 수립 개선.

**HTTP의 진화는 "더 빠른 버전"의 역사가 아니라,
"이전 병목을 해결해온 과정"이다.**

REFERENCES

# 참고 자료


RFC 9112 — HTTP/1.1

RFC 9113 — HTTP/2

RFC 9114 — HTTP/3

RFC 9000 — QUIC

MDN — Connection management in HTTP/1.x

MDN — HTTP/3 · Head-of-line blocking

Cloudflare — HTTP/2 vs HTTP/3

web.dev — Performance & protocols